

Phi-4 경량화 — 구조적 프루닝 방법 비교

Phi-4 (14B) → Jetson Orin Nano 8GB 온디바이스 구동을 위한 structured pruning 실험

Pseudo-Lab · LLM-VLM-in-Jetson · 2026-06 · 실험 GPU: NVIDIA RTX PRO 6000 Blackwell 96GB (GCP)

1. 배경 & 목표

목표는 **Phi-4 (14.7B)**를 압축해 **Jetson Orin Nano 8GB**에서 한국어 에이전트로 구동하는 것이다. Jetson Orin Nano는 CPU/GPU가 8GB를 공유하고 OS 런타임이 1.5~2GB를 점유해, 실사용 가능 메모리는 약 5.5~6.5GB다.

구성	가중치 메모리	비고
Phi-4 (14.7B) FP16	~29 GB	원본
INT4 (4-bit) 양자화만	~7.3 GB	8GB에 빠듯 / OOM 위험

즉 **양자화만으로는 부족**하며, 파라미터 차원 자체를 줄이는 **구조적 프루닝**이 필요하다. 배포 런타임은 엣지에 실용적인 llama.cpp (GGUF)를 선택했고, 이는 dense 연산이라 **실제 차원 축소(structured)만 메모리 절감으로 이어진다**(마스킹/sparsity는 무의미).

2. 방법론 — 두 축 × 중요도 지표

구조적 프루닝은 "무엇을 자르나(축)" × "어떻게 중요도를 매기나(지표)"로 정리된다.

축	자르는 단위	대표 방법
Width (폭)	MLP intermediate 뉴런 / attention head / hidden 차원	Minitron, LLM-Pruner, SliceGPT
Depth (깊이)	transformer 레이어/블록 통째	ShortGPT

중요도 지표	핵심	대표
활성값 크기 (activation)	보정 데이터에서의 실제 반응 크기 (forward만)	Minitron ← 본 실험
Weight magnitude	가중치 절댓값 W	베이스라인
Block Influence	레이어 입력·출력 코사인 (1 - cos)	ShortGPT ← 본 실험
PCA / 회전불변	주성분으로 hidden 차원 슬라이스	SliceGPT

3. 구현 — 라이브러리 대신 직접 구현

구조적 프루닝 라이브러리를 시도했으나 Phi-4(transformers 5.x)에 막혀, 핵심 알고리즘을 **PyTorch로 직접 구현**했다.

라이브러리	결과
Torch-Pruning (의존성 그래프 자동)	✗ transformers 5.x Phi-4 추적 중 무한루프 폭주(38분 CPU 100%)
LLM-Pruner	✗ torch-pruning 기반(동일 폭주, CPU·GPU 둘 다 확인) + Phi-4 미지원 (LLaMA계열만)

SliceGPT (microsoft/TransformerCompression)	✓ Phi-4 동작 (phi3_adapter 이름매칭만 patch, transformers 4.41)
직접 구현 (Minitron식 width, ShortGPT식 depth)	✓ Phi-4에 바로 동작, 버전·하드웨어 무관

핵심: 프루닝은 마스킹(0 채움)이 아니라 슬라이싱(텐서 인덱싱으로 차원 실제 축소)으로 구현 → 파라미터 수가 실제로 줄어 GGUF/메모리에 직결. Llama(분리 gate/up)·Phi-3/4(fused gate_up_proj) 구조 모두 지원.

4. 실험 환경

- 모델: microsoft/phi-4 (14.66B, Phi-3 아키텍처)
- 보정셋: MarkrAI/KoCommercial-Dataset (한국어) — 활성값 중요도 측정용
- 평가: KMMLU (lm-eval, 한국어 객관식) · 4-bit 메모리 추정
- GPU: RTX PRO 6000 Blackwell 96GB (GCP g4-standard-48)

5. 결과 — 방법별 비교

KMMLU limit=20 (빠른 동일-run 비교). 감축률 = 전체 파라미터 기준.

방법	축	중요도	감축률	4bit 메모리	KMMLU	원본 대비
원본 Phi-4	—	—	—	7.33 GB	34.11%	100%
depth (ShortGPT)	depth	Block Influence	27.9%	5.28 GB	31.89%	~94%
width (Minitron)	width	활성값	22.5%	5.68 GB	18.33%	~54%
width (magnitude)	width	W	22.5%	5.68 GB	7.33%	~21%

핵심 발견

- ① depth(ShortGPT)가 압도적 — 더 많이 줄였는데(27.9%) 정확도는 거의 안 떨어짐(34.11→31.89, -2.2점, **94% 보존**). 잉여 레이어를 제거하기 때문.
- ② 활성값 ≫ magnitude — 같은 30% MLP 프루닝인데 18.33% vs 7.33%로 **2.5배** 차이. "어떤 뉴런을 남기냐"가 결정적 → Minitron이 활성값을 쓰는 이유를 정량 입증.

SliceGPT (공식 라이브러리) — 또 하나의 width 프루닝

SliceGPT는 hidden 차원을 PCA로 슬라이스. 지표가 wikitext2 PPL이라 위 KMMLU와는 별개(직접 비교 불가). 회복 파인튜닝 전 수치.

모델	sparsity	파라미터	PPL (원본→슬라이스)
Phi-3-mini	25%	3.82B → 3.24B (15.3%↓)	6.01 → 9.30
Phi-4	30%	14.66B → 11.48B (21.7%↓)	6.46 → 9.15

SliceGPT는 공식 lib(microsoft/TransformerCompression)로 Phi-4 정상 동작 확인. (어댑터 이름매칭 patch만 필요)

6. Block Influence 분석 — 어떤 레이어가 잉여인가

각 레이어가 잔차 스트림을 얼마나 바꾸는지($BI = 1 - \cos(\text{입력}, \text{출력})$)를 측정. 낮을수록 잉여.

레이어 구간	BI	해석
0 (첫 레이어)	0.594	압도적 1위 — 임베딩→첫 변환, 가장 중요
1~11	0.10~0.25	중요
27~37 (중간~후반)	0.03~0.05	거의 0 — 잉여 → 제거 대상
38 (끝 직전)	0.144	다시 중요 — 출력 준비 → 유지

→ "중간~후반 레이어가 잉여하다"는 ShortGPT 논문의 주장을 Phi-4에서 직접 재현.

7. 개념 정리 — structured / masking 구분

메모리 절감은 "structured + slicing" 한 칸에서만 일어난다.

	Masking (0 채움, shape 유지)	Slicing (제거, shape 축소)
Unstructured (개별 가중치)	Wanda / SparseGPT	불가
Structured (채널·레이어)	torch.nn.utils.prune (ln_structured)	Minitron · ShortGPT (본 실험)

Wanda를 쓰지 않은 이유: Wanda는 unstructured 마스킹이라 차원이 안 줄어든다. 배포 런타임 llama.cpp(dense)는 sparsity를 활용 못 해 메모리·속도 이득이 0. (2:4 + TensorRT/vLLM 같은 sparse 런타임에서만 의미)

8. 결론 & 다음 단계

- 구조적 프루닝만으로 4bit 5.3GB 달성 → 이미 Jetson 8GB 적재 가능 수준.
- Phi-4 → Jetson엔 depth(ShortGPT)가 유리 — 더 많이 줄이고 정확도 손실 최소, distillation 회복 부담도 적음.
- 모든 수치는 distillation 회복 전 값. 다음 단계로 distillation(teacher=Phi-4, 한국어)으로 width 하락 회복 → GGUF 양자화 → Jetson 실측 예정.

caveat: KMMLU limit=20은 빠른 비교용 소표본(노이즈 존재). 방법 간 상대 비교는 견고하나 절대값은 full 측정 시 변동. 동일-run 델타가 공정 비교 기준.